

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a1cd7e4943c878cce.jsonl`  
- SHA-256: `73fafd1a9c48338a37ae271b20cd4c2018f7eba1b245a61903812a23ce8b4f55`  
- Source modified: `2026-07-02T02:52:58+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:47:11.980Z)

CONTEXT (read carefully):

- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: triage all 10 open khelsutra issues against current code. Repo:

C:/Users/avidu/Projects/khelsutra-guru/khelsutra.

gh issue list --state open, then for EACH issue (#72 #97 #98 #99 #101 #102 #103 #111 #112 #114) read it (gh issue view N) and verify against current web/ + site/ code whether it is still valid, partially fixed, or fully fixed-but-open (e.g. #103 video stats - #140 just landed fps in the library; check what remains). Classify each: quick-win (S effort, Claude-doable), real-feature (M/L), owner-gated (e.g. #114 CF Access dashboard, #97 topology decision). Report stale-open-issues as hygiene findings and genuinely-valuable quick wins as P1/P2.

2. user (2026-07-02T02:47:18.859Z)

f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)

master...origin/master

3. user (2026-07-02T02:47:20.701Z)

```
114      OPEN      Cloudflare Access: an allowlisted email still couldn't sign in (alpha auth)
2026-06-23T10:25:52Z
112      OPEN      Share to WhatsApp sends only a text message, not the reel video bug
2026-06-22T18:27:52Z
111      OPEN      Resilient processing UX: survive page refresh + continually-updating progress
enhancement      2026-06-22T18:15:04Z
103      OPEN      SPA: show video stats (name, size, fps, resolution, duration) + persist them on
reload           2026-06-22T14:18:41Z
102      OPEN      SPA: &debug=1 (trace panel) / &debug=2 (full runlog) URL params for debugging +
tracing          2026-06-22T14:16:43Z
101      OPEN      SPA: show a friendly failure notice on a timeout/5xx (Cloudflare 524), not the
raw Cloudflare error HTML      2026-06-22T14:14:16Z
99       OPEN      SPA: keep the uploaded video's filename visible during processing (it disappears
mid-job)         2026-06-22T14:01:52Z
98       OPEN      SPA: replace the demo rally-selection the moment an upload/processing starts (so
the user knows their reel is coming)      2026-06-22T13:57:56Z
97       OPEN      Hosted alpha: split-origin (Pages app. + tunnel api.) breaks Cloudflare Access
for the SPA ? single-origin needed      2026-06-22T13:54:39Z
72       OPEN      i18n: add Telugu (te) locale (catalog + Noto Telugu font, key-parity green)
2026-06-21T18:17:34Z
```

4. user (2026-07-02T02:47:34.364Z)

```
=== ISSUE #72 ===
title: i18n: add Telugu (te) locale (catalog + Noto Telugu font, key-parity green)
state: OPEN
author: avidullu (Avi)
labels:
comments:      0
assignees:
projects:
milestone:
number: 72
--
```

Add Telugu (`te`) to the supported locales

Proposal: add **Telugu (`te`)** as a first-class locale, alongside the shipped **en**, **kn**, **hi**, **es**, **da**, **id** set.

Why: Telugu (Andhra Pradesh + Telangana) is one of the largest Indian languages and a strong badminton region ? a natural extension of the **en**/**kn**/**hi** India foundation, broadening the home-market reach before the SE-Asia/EU/East-Asia locales.

Fit with the existing architecture (reuses the foundation verbatim ? no re-architecture):

- Add **te** to **SUPPORTED_LOCALES** (**web/src/i18n/index.ts**) + a reviewed **locales/te/common.json** sibling in the shared repo-root catalog (**D-CATALOG-HOME**). Always-fall-back-to-**en** unchanged.
- **Font:** Telugu is non-Latin ? self-host **OFL Noto Telugu** via **@fontsource**, **unicode-range**-scoped, exactly like the **kn**/**hi** pattern (**ED2 / D7**). One small-to-medium OFL font; **no new licensing decision**. This makes it a "Phase B-shaped" add (catalog + one Noto font), like Thai ? not a CJK-class effort.
- **Plurals:** Telugu CLDR cardinal is **one**, other (2-form, like English/Danish) ? **no many-arm wrinkle** as in Spanish. Trivial for the catalog/tooling.
- **CI:** key-parity + no-bare-strings stay merge gates ? the **te** catalog must match the **en** key set exactly or CI fails.

- **Negotiation test** (per FRIENDLY §6.5): ``navigator.languages=["te-IN"]` ? `te``; region subtags normalize; a missing key falls back to ``en``. Auto-selection proven by a test, not assumed.

- Once ``site/`` is trilingual (B3), extend the ``t()/`data-i18n`` shim + ``Accept-Language: te`` negotiation the same way.

Translation pipeline: Gemini draft ? **native Telugu-speaker review** (the public-launch gate, ED3/EX1). AI drafts are inference-only, never the source of truth.

Docs: track under ``docs/MULTILINGUAL_EXPANSION.md`` (peer of ``FRIENDLY_AND_MULTILINGUAL.md``); pattern follows Phase A (#67) and the D-CATALOG-HOME hoist (#70).

Cross-repo: the public **``rally-annotator``** sister repo (the open labeling tool) is also targeting ``te`` in its own localization, so adding it here keeps the two surfaces aligned on the same locale set and catalog keys.

=== ISSUE #97 ===

title: Hosted alpha: split-origin (Pages app. + tunnel api.) breaks Cloudflare Access for the SPA ? single-origin needed

state: OPEN

author: avidullu (Avi)

labels:

comments: 0

assignees:

projects:

milestone:

number: 97

--

Symptom

On the hosted alpha, the SPA loaded at ``app.khelsutra.guru`` but **every** upload failed "Can't reach KhelSutra. Is it running?" ? even though the engine was healthy.

Root cause (NOT file size)

``app.khelsutra.guru`` (Cloudflare Pages) and ``api.khelsutra.guru`` (cloudflared tunnel) are **different origins**. The SPA's ``fetch`` (``web/src/api/client.ts:60``) sends **no** ``credentials: 'include'``, so cross-origin calls to ``api.`` never carry the Cloudflare Access cookie ? Access ``302``'s every API call to the login page ? the fetch rejects ? "can't reach." It fails on **any** upload, not just large ones.

Verified it's not size: a real **95 MiB** chunk PUT through the tunnel returned `200` in **~2s** (and 50 MiB ? `200`).

Fix applied (2026-06-22) ? single-origin

The engine already bundles the SPA (P1b), so I repointed to **single-origin**:

``api.khelsutra.guru`` now serves the **SPA** at ``/`` and the **API** at ``/api`` behind **one** Access cookie. Verified end-to-end via a service token: ``GET /` ? 200` (``<title>KhelSutra Studio</title>``), ``/api/health` ? 200`, ``upload/init`` + 95 MiB + 50 MiB chunk PUTs ? `200`.

Remaining / to pick up

- ``app.khelsutra.guru` DNS** needs repointing to the tunnel: ``CNAME app`` 55142398-732c-4803-8a9c-48766f1f5cac.cfargotunnel.com, proxied. Blocked on the CF API token lacking ``Zone.DNS:Edit`` (the scope add didn't take); do via dashboard or fix the token. (The Pages custom domain for ``app.`` was removed.)
- Update** ``deploy/DEPLOY_ALPHA.md`` to recommend **single-origin** (engine-served SPA) as the default for the SPA+Access case, OR document the cross-origin requirements if split is desired: SPA ``credentials: 'include'`` + engine CORS ``allow_credentials=true`` + a way to pre-warm the ``api.`` Access cookie (the hard part for fetch).
- Refresh** the engine's bundled SPA (currently commit #68) to latest, so the localized reel-watermark (LW2 #73) + recent polish (#82/#83/#86) ship in the hosted app.
- Lesson:** a service-token full-chain test passes (header auth) while a browser cookie path fails ? the hosted CUJ must be verified with a **real browser session**.

=== ISSUE #98 ===

title: SPA: replace the demo rally-selection the moment an upload/processing starts (so the user knows their reel is coming)

state: OPEN

author: avidullu (Avi)

labels:

comments: 0

assignees:

projects:

milestone:

number: 98

--

****Feedback from the first real hosted-alpha use.****

When a user starts an ingest via **any** channel (bucket `gs://` or local file upload), the **demo rally-selection** shown below the middle fold should **change immediately** ? to a processing/loading state for **their** video ? instead of continuing to show the static demo rallies.

Why: right now the user kicks off an upload but the area below still shows demo content, so it's not obvious that **a new reel is being generated for them**. The moment processing starts, that section should flip to "your reel is processing?" (with the job progress), so the path from upload ? their reel is visually continuous.

Acceptance: starting an upload/ingest immediately clears/replaces the demo selection with a per-job processing state; on completion it becomes the real rally picker for the uploaded video.

=== ISSUE #99 ===

title: SPA: keep the uploaded video's filename visible during processing (it disappears mid-job)

state: OPEN

author: avidullu (Avi)

labels:

comments: 0

assignees:

projects:

milestone:

number: 99

--

****Feedback from the first real hosted-alpha use.****

While a video is **processing**, its **filename disappears** from the UI. It should stay visible (pinned in the processing card/header) for the whole job, so the user always knows **which** video is being worked on ? especially important because processing takes several minutes on a CPU box.

Part of the "processing-state UX" cluster with #98 (demo selection should flip to a per-job processing state on upload). Acceptance: the uploaded filename remains visible from upload ? processing ? result.

=== ISSUE #101 ===

title: SPA: show a friendly failure notice on a timeout/5xx (Cloudflare 524), not the raw Cloudflare error HTML

state: OPEN

author: avidullu (Avi)

labels:

comments: 0

assignees:

projects:

milestone:

number: 101

--

****From the first real hosted reel build.**** On a Cloudflare **524** (origin timeout) **during "Build reel"**, the SPA surfaced the **raw Cloudflare error-page HTML** ("Build failed" with a wall of `<!DOCTYPE html>?524?`). Ugly and alarming ? **especially** because the reel actually built (the 524 only timed out the HTTP response).

Fix: the SPA error mapping (`lib/errors.ts` / `errorText.ts`, the A5 layer) should detect a **non-JSON / 5xx / 524 / timeout** response and show a **friendly message** instead of rendering the body ? e.g. **"This is taking longer than usual ? your reel may still be building. We'll have async builds soon; for now, try again in a moment."** Bonus: on a compile timeout, **probe for the finished reel** (`/api/highlights/{video\_id}/{name}`) before declaring failure.

Relates to engine #329 (async `/api/compile` removes the 524 at the source); this is the client-side defense regardless. Same "loud-but-friendly failures" bar as A5.

5. user (2026-07-02T02:47:40.010Z)

=== ISSUE #102 ===

title: SPA: &debug=1 (trace panel) / &debug=2 (full runlog) URL params for debugging + tracing
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 102
--

Idea (owner)

A `&debug=` ladder on the SPA URL so debugging/tracing is well-covered:

- `?video=?&debug=1` ? surface **tracing** (a debug panel)
- `?video=?&debug=2` ? the **entire runlog file** for that video/job

Ground-truth mapping (what already exists ? so this is mostly a surfacing layer)

- The SPA is already `?video=?`-driven and threads an **`x-correlation-id`** on every call.
- **GET /api/jobs/{job_id}** already returns **status**, **progress**, **result**, and **reports** (the per-video obsreport paths).
- The engine emits a **per-video `obsreport` / `report.json`** next to each processed video.
- A dedicated per-video trace endpoint is proposed in #328.

Proposed design

- **`&debug=1`** ? in-UI trace panel: the job's phase timeline + timings + chunk/segment counts + the correlation id, pulled from **/api/jobs/{id}** (and the #328 trace endpoint). Light, low-risk.
- **`&debug=2`** ? fetch/download the full runlog: the structured obsreport for the job (+ optionally the raw engine log lines for that **cid**).

Security (important on the hosted / Cloudflare-Access deploy)

- **Scope strictly to the signed-in user's own** job/video ? **?video=X&debug=2** must not dump someone else's run.
- **Redact** secrets + sensitive local paths before surfacing the runlog.
- Consider gating **`&debug=2`** (raw runlog) to the **operator/owner**; **`&debug=1`** (trace panel) is fine for any authed user on their own job.

Relates to: engine #328 (per-video runlog endpoint, the data source) + engine #326 (request tracing/replay). This issue is the **user-facing `&debug=` access layer** on top of those.

=== ISSUE #103 ===

title: SPA: show video stats (name, size, fps, resolution, duration) + persist them on reload
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 103
--

Feedback from real use. On reload the uploaded **filename isn't shown**, and it'd be good to

show **stats** about the video **? name, size, fps, resolution, duration**.

Extends #99 (keep the filename visible during processing) with a richer **"video info" panel**.

Ground truth

The engine already probes the video during the validation phase (duration via ``get_video_duration``; `ffprobe` in the validators), so the metadata exists **? it just isn't surfaced**.

- **Engine**: expose video metadata (name, bytes, fps, WxH, duration, codec) via ``/api/videos/{video_id}`` or on the job ``result`` (it's stored per ``video_id``).
- **SPA**: render a compact video-info panel, shown from upload **? processing ? result**, and **persisted across reload** (keyed by ``video_id``, like the rallies).

=== ISSUE #111 ===

title: Resilient processing UX: survive page refresh + continually-updating progress

state: OPEN

author: avidullu (Avi)

labels: enhancement

comments: 0

assignees:

projects:

milestone:

number: 111

--

Context (from the first live alpha run on the droplet, 2026-06-22). A real ~8.3-min upload took ~5+ min of Gemini segmentation (7 chunks of 90s). During that window the SPA shows a coarse, near-static "processing" phase, and a **page refresh** drops the user back to the ingest form even though the job keeps running server-side. Users naturally refresh when nothing visibly moves **? and then think they've lost their work**.

Already fine (do NOT re-solve)

The **backend** is refresh-safe: jobs run server-side and are persisted in the SQLite ``jobs`` table (the async job worker); a client refresh does **not** kill or affect the running job.

The gaps

1. **SPA** loses the in-flight ``job_id`` on refresh. ``web/src/hooks/useIngestJob.ts`` holds the active ``job_id`` only in the poll-loop's runtime state **? there's no ``localStorage``/URL persistence**. On refresh the client can't reconnect to the still-running job **? the user lands back on the ingest form**. (On `*done*` it sets ``?video=<id>``, but mid-job there's nothing to resume from.)
2. **Progress** looks static for minutes. The SPA reflects ``job.progress`` (a coarse free-text phase), but the Gemini segmenter `*logs*` per-chunk progress ("Extracting Chunk 3/7") and never pushes it to the job-progress hook **? so the phase text doesn't move during the multi-minute segmentation**.

Requirements / acceptance

- [] **Survive refresh** **? persist the active ``job_id`` (URL ``?job=<id>`` and/or ``localStorage``)**; on load, if a job is in-flight, **resume polling it** and reconnect to live progress, then hand off to the existing ``?video=`` flow on done. A refresh during processing must seamlessly re-attach to the running job, never lose it.
- [] **Continually-updating, reassuring progress** **? a heartbeat / elapsed-time indicator + an animated state** so the user always sees movement even when the engine phase is unchanged. Copy along the lines of "This can take a few minutes **? it's safe to leave or refresh this tab; you won't lose your place**."
- [] **(Engine, linked ? repo ``Khelsutra/badminton-highlight-indexer``)** Report granular progress from the Gemini segmenter via the job-progress hook (e.g. ``segmenting chunk 3/7``) so the SPA has real movement to show. Small change in ``backend/pipeline/segmenters/gemini.py``.

Nice-to-haves

- Show partial results as chunks complete.
- A Cancel button.
- Surface the active/recent job in history so a returning user can find it.

Surfaced live while watching the first real alpha run **? a long Gemini job + a refresh-fragile**

client is the most likely way an early tester gets confused or "loses" a working job._

=== ISSUE #112 ===

title: Share to WhatsApp sends only a text message, not the reel video

state: OPEN

author: avidullu (Avi)

labels: bug

comments: 0

assignees:

projects:

milestone:

number: 112

--

Reported from live alpha use (2026-06-22): clicking **Share to WhatsApp** on a finished reel opens WhatsApp with a **pre-filled text message but no video** ? the user expects the reel itself to be shared.

Current behavior

``web/src/components/CompileResult.tsx:79`` builds a ``https://wa.me/?text=<message>`` link. As the in-code comment notes, ``wa.me`` can carry **text only** ? it can't attach the local file ? so the user is silently left to manually attach the just-downloaded reel. Most users read "Share" as "share the video," so this reads as broken.

Why a **link** won't fix it either

The reel lives at ``GET /api/highlights/{video_id}/{filename}``, which is **behind Cloudflare Access** ? a recipient outside the tenant can't open a shared link. So the fix is to share the **file**, not a URL.

Proposed fix ? Web Share API with the file

- Fetch the reel as a blob (the SPA holds the CF-Access session): ``fetch(highlightUrl(...))`` ? ``blob`` ? ``new File([blob], build.filename, { type: "video/mp4" })``.
- If ``navigator.canShare?.({ files })`` ? ``navigator.share({ files: [file], title, text: t("result.shareText") })`` ? on supported platforms (mobile Chrome/Safari, capable desktops) this hands the **actual video** to WhatsApp and any other share target.
- **Graceful fallback** where file-share is unsupported (many desktops): keep a download + a clear hint ("Reel downloaded ? attach it in WhatsApp"), or the current ``wa.me`` text link. Never a dead/empty share.

Acceptance

- [] On a file-share-capable browser (mobile especially), Share hands the **reel video** to WhatsApp/other targets ? not just text.
- [] On unsupported browsers, a graceful, clearly-explained fallback (download + "attach it" guidance).
- [] Localized share text (``result.shareText``) retained.

Honest scope note

Desktop browsers ? including **Edge**, where this was hit ? have limited Web-Share-**file** support, so they may still fall back to download-then-attach. The real win is **mobile + capable browsers getting a true one-tap video share** (which is where most reels get shared anyway). A public/signed shareable reel URL (so a plain link works for anyone) is a separate, privacy-sensitive feature worth its own issue if you want link-sharing too.

=== ISSUE #114 ===

title: Cloudflare Access: an allowlisted email still couldn't sign in (alpha auth)

state: OPEN

author: avidullu (Avi)

labels:

comments: 0

assignees:

projects:

milestone:

number: 114

--

Symptom (real demo, 2026-06-23)

Demoing the alpha (`app.khelsutra.guru`) to a friend: his email **was whitelisted** in the Cloudflare Access policy, but he **could not sign in** (the email-OTP / Access login didn't let him through), so he couldn't use the app with his own address. The owner's own email works fine.

The core flow is great ? small clip ? highlights was "pure bliss." This is purely the Access allowlist not admitting a second, whitelisted address.

Context (what's deployed)

- CF Access app covering `app.` + `api.khelsutra.guru`, team `khelsutra.cloudflareaccess.com`, email-OTP login gated to an **allowlist** policy.
- Per the deploy notes the policy was created allowing a single email (`avi.dullu@gmail.com`); adding a guest means the guest's email must be in **that app's** policy include rule.
- Background: `deploy/DEPLOY\_ALPHA.md` (the Access/tunnel setup).

Likely causes to check (in order)

1. **Was the email added to the right policy?** The include rule must be on the Access **application** that covers `app.khelsutra.guru` (not a different app), and the policy action must be **Allow**.
2. **Include selector type** ? "Emails" (exact addresses) vs "Emails ending in" (domain). A typo / trailing space / case in the exact-email entry will silently fail to match.
3. **Policy actually saved + propagated** ? confirm the edit was saved on the app and re-test in a fresh/incognito session (Access caches the session cookie).
4. **OTP delivery** ? did the friend receive the one-time PIN email at all? (spam folder / wrong address / provider blocking).
5. **Single-origin note** ? the working path is `api.khelsutra.guru` (the engine serves the SPA single-origin); confirm the guest used that, not a stale `app.` Pages URL with the cross-origin cookie issue (see issue history).

Repro

1. Add `guest@example.com` to the Access app's Allow policy (Emails ? exact).
2. From a clean browser, open the app URL ? enter `guest@example.com` ? request OTP.
3. Expected: PIN email arrives ? entering it admits the guest. Actual: login fails / not admitted.

Severity / priority

? Not blocking (owner can use it; demos work by driving it yourself) but **blocks sharing with whitelisted guests** ? needed before inviting alpha testers. Deferred by owner 2026-06-23.

6. assistant (2026-07-02T02:47:58.489Z)

Now let me verify each issue against current code. First, the repo structure.

7. user (2026-07-02T02:48:00.188Z)

```
NEXT_STEPS.md
README.md
deploy
docs
locales
node_modules
site
web
---
App.tsx
annotator
api
components
hooks
i18n
```

```
index.css
lib
main.tsx
test
vite-env.d.ts
---
web/src/api:
client.test.ts
client.ts
locale.test.ts
types.ts
upload.test.ts

web/src/components:
AnnotatePanel.test.tsx
AnnotatePanel.tsx
CompileResult.tsx
FfmpegBanner.test.tsx
FfmpegBanner.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts
QueryBar.test.tsx
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.test.tsx
SelectionFooter.tsx
SetupWizard.test.tsx
SetupWizard.tsx
StatusChip.tsx
TapFilters.test.tsx
TapFilters.tsx
VideoLibrary.test.tsx
VideoLibrary.tsx
a4.touch.test.tsx
a8.status.test.tsx
build-error.test.tsx
upload.test.tsx

web/src/hooks:
useIngestJob.test.ts
useIngestJob.ts
useRallySelection.ts
useUpload.test.ts
useUpload.ts

web/src/lib:
errorText.test.ts
errorText.ts
errors.test.ts
errors.ts
floor.test.ts
floor.ts
format.test.ts
format.ts
log.ts
query.test.ts
query.ts
selection.test.ts
selection.ts
video.test.ts
video.ts
```

```

da
en
es
hi
id
kn
catalog-invariants.test.ts
hygiene.test.ts
i18n.test.tsx
index.ts
25:export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
26:export type Locale = (typeof SUPPORTED_LOCALES)[number];
52:    supportedLngs: [...SUPPORTED_LOCALES],

```

9. user (2026-07-02T02:48:10.235Z)

```

1      // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2      // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3      //
4      // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";
11     import { classifyError, type IngestError } from "../lib/errors";
12     import { log, newCorrelationId } from "../lib/log";
13     import type { JobStatus } from "../api/types";
14
15     const POLL_MIN_MS = 1000;
16     const POLL_MAX_MS = 5000;
17     const POLL_BACKOFF = 1.5;
18     const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20     export type IngestPhase =
21       | { status: "idle" }
22       | { status: "submitting" }
23       | { status: "polling"; progress: string | null }
24       | { status: "done"; videoId: string }
25       | { status: "error"; error: IngestError };
26
27     interface Run {
28       cancelled: boolean;
29       timer?: ReturnType<typeof setTimeout>;
30     }
31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33       const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34       const runRef = useRef<Run | null>(null);
35       // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36       const onLoadedRef = useRef(onLoaded);
37       onLoadedRef.current = onLoaded;
38
39       const stop = useCallback(() => {
40         const run = runRef.current;
41         if (run) {
42           run.cancelled = true;

```

```

43         if (run.timer) clearTimeout(run.timer);
44     }
45     runRef.current = null;
46 }, []));
47
48 // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
49 // flag idiom).
50 useEffect(() => stop, [stop]);
51
52 const submit = useCallback(
53   (rawUri: string, locale?: string, compute?: string) => {
54     const uri = rawUri.trim();
55     if (!uri) {
56       setPhase({ status: "error", error: { code: "invalidUri" } });
57       return;
58     }
59     stop(); // supersede any prior run
60     const run: Run = { cancelled: false };
61     runRef.current = run;
62
63     const cid = newCorrelationId();
64     const startedAt = Date.now();
65     setPhase({ status: "submitting" });
66     log("info", "ingest.submit_start", { cid, uri });
67
68     const fail = (error: IngestError, event: string) => {
69       if (run.cancelled) return;
70       setPhase({ status: "error", error });
71       const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
72 : "error";
73       log(level, event, { cid, uri, code: error.code, detail: error.detail });
74     };
75
76     const poll = (jobId: string, delay: number) => {
77       run.timer = setTimeout(() => {
78         void (async () => {
79           if (run.cancelled) return;
80           let job: JobStatus;
81           try {
82             job = await getJob(jobId, cid);
83           } catch (err) {
84             fail(classifyError(err), "ingest.poll_error");
85             return;
86           }
87           if (run.cancelled) return;
88
89           if (job.status === "done") {
90             const videoId = job.video_id ?? job.result?.video_id ?? null;
91             // Two status layers: a `done` job with a `failed` pipeline verdict (or no
92             // video_id) is
93             // a success-shaped FAILURE ? never load it, never hang.
94             if (job.result?.status === "failed" || !videoId) {
95               fail({ code: "emptyResult" }, "ingest.empty_result");
96               return;
97             }
98             setPhase({ status: "done", videoId });
99             log("info", "ingest.done", { cid, uri, video_id: videoId });
100             onLoadedRef.current?.(videoId);
101             return;
102           }
103           if (job.status === "failed") {
104             fail({ code: "engine", detail: job.error ?? "job failed" },
105               "ingest.job_failed");
106             return;

```

```

104         }
105         // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106         setPhase({ status: "polling", progress: job.progress ?? null });
107         if (Date.now() - startedAt > POLL_CEILING_MS) {
108             fail({ code: "timeout" }, "ingest.poll_timeout");
109             return;
110         }
111         poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112     }));
113     }, delay);
114 };
115
116 void (async () => {
117     let jobId: string;
118     try {
119         const resp = await processVideo(uri, cid, locale, compute);
120         jobId = resp.job_id;
121     } catch (err) {
122         fail(classifyError(err), "ingest.submit_error");
123         return;
124     }
125     if (run.cancelled) return;
126     log("info", "ingest.queued", { cid, uri, job_id: jobId });
127     setPhase({ status: "polling", progress: null });
128     poll(jobId, POLL_MIN_MS);
129 }));
130 },
131 [stop],
132 );
133
134 const cancel = useCallback(() => {
135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138 }, [stop]);
139
140 const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143 }, [stop]);
144
145 const busy = phase.status === "submitting" || phase.status === "polling";
146 return { phase, submit, cancel, reset, busy };
147 }
148

```

10. user (2026-07-02T02:48:10.764Z)

```

1     import { useEffect, useMemo, useState } from "react";
2     import { useTranslation, Trans } from "react-i18next";
3     import { useRallySelection } from "../hooks/useRallySelection";
4     import { RallyGrid } from "../components/RallyGrid";
5     import { QueryBar } from "../components/QueryBar";
6     import { TapFilters } from "../components/TapFilters";
7     import { SelectionFooter } from "../components/SelectionFooter";
8     import { LanguageSwitcher } from "../components/LanguageSwitcher";
9     import { CompileResult, type BuildState } from "../components/CompileResult";
10    import { IngestPanel } from "../components/IngestPanel";
11    import { FfmpegBanner } from "../components/FfmpegBanner";
12    import { VideoLibrary } from "../components/VideoLibrary";
13    import { AnnotatePanel } from "../components/AnnotatePanel";
14    import { SetupWizard, isWizardDone } from "../components/SetupWizard";
15    import { runQuery, type ParsedQuery } from "../lib/query";
16    import { floorRisk } from "../lib/floor";

```

```

17     import { reelFilename } from "./lib/format";
18     import { classifyError } from "./lib/errors";
19     import { errorText } from "./lib/errorText";
20     import { log, newCorrelationId } from "./lib/log";
21     import { ApiError, compileReel, getConfig, highlightUrl, listRallies, listVideos,
minShotCount } from "./api/client";
22     import { videoId as videoIdOf, videoName } from "./lib/video";
23     import type { EngineConfig, RallySegment } from "./api/types";
24
25     /** One compact, structured description of how a query was understood (for the trace
log). */
26     function parseTrace(q: ParsedQuery) {
27         return {
28             raw: q.raw,
29             recognized: q.recognized,
30             filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
31             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
32             limit: q.limit ?? null,
33             unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
34         };
35     }
36
37     // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
38     // contract; only honest fields are surfaced. A real session loads from POST /api/query.
39     const DEMO_RALLIES: RallySegment[] = [
40         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
41         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
42         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
43         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
44         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
45         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
46     ];
47
48     type LoadState =
49         | { status: "demo" }
50         | { status: "loading" }
51         | { status: "loaded"; count: number }
52         | { status: "error"; error: string };
53
54     /** Annotate-in-UI (P8) is behind a runtime feature flag ? default OFF (D13: ships gated
until consented
55     * / no-minors footage is in hand). Turn it on per-browser with `?annotate=1` or
56     * `localStorage['khelsutra.annotate'] = '1'`. No build step; the owner enables it when
ready. */
57     function annotateFlag(): boolean {
58         try {
59             if (new URLSearchParams(window.location.search).get("annotate") === "1") return
true;
60             return localStorage.getItem("khelsutra.annotate") === "1";
61         } catch {
62             return false;
63         }
64     }
65
66     export default function App() {
67         const { t, i18n } = useTranslation();
68         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,

```

```

69      // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
70      // without a reload (it also rewrites ?video= so a refresh is stable).
71      const [videoId, setVideoId] = useState<string | null>{
72        () => new URLSearchParams(window.location.search).get("video"),
73      };
74
75      const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
76      const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
77      const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
78      const [reelName, setReelName] = useState("highlights");
79      const [build, setBuild] = useState<BuildState>({ status: "idle" });
80      // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
81      const [wizardDone, setWizardDone] = useState(isWizardDone);
82      // The human name of the loaded video ? shown PERSISTENTLY so the operator always
knows which match
83      // they're editing. Once a video loads, the library + ingest panel (which showed the
name) unmount,
84      // so without this the name would vanish. Set from the library on click; else resolved
below.
85      const [videoLabel, setVideoLabel] = useState<string | null>(null);
86      // P8 (flagged): switch the loaded-video view between picking rallies and annotating
them.
87      const [annotateEnabled] = useState(annotateFlag);
88      const [view, setView] = useState<"pick" | "annotate">("pick");
89
90      // Display order is separate from selection: a query can re-sort the cards while the
shared
91      // selection set (sel) stays the single source of truth for what lands in the reel.
92      const sel = useRallySelection(rallies, "all");
93
94      // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
95      useEffect(() => {
96        if (!videoId) return;
97        const cid = newCorrelationId();
98        log("info", "rallies.load_start", { cid, video_id: videoId });
99        // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
100        // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
101        setLoadState({ status: "loading" });
102        let alive = true;
103        listRallies(videoId, cid)
104          .then((rs) => {
105            if (!alive) return;
106            setRallies(rs);
107            setOrder(rs);
108            sel.apply(rs.map((r) => r.id));
109            setLoadState({ status: "loaded", count: rs.length });
110            log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
111          })
112          .catch((err) => {
113            if (!alive) return;
114            // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
115            // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
116            const e = classifyError(err);
117            setLoadState({ status: "error", error: errorText(t, e) });
118            log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
119          });
120        return () => {
121          alive = false;

```

```

122     };
123     // videoId is stable for the page; sel.apply is a stable callback.
124     // eslint-disable-next-line react-hooks/exhaustive-deps
125     }, [videoId]);
126
127     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
128     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
129     const [cfg, setCfg] = useState<EngineConfig | null>(null);
130     useEffect(() => {
131         let alive = true;
132         getConfig()
133             .then((c) => alive && setCfg(c))
134             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
135         return () => {
136             alive = false;
137         };
138     }, []);
139     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
140
141     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
142     const ordinalOf = useMemo(() => {
143         const m = new Map<number, number>();
144         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
145         return (id: number) => m.get(id) ?? 0;
146     }, [rallies]);
147
148     // Load a video into the picker ? from a fresh ingest (IngestPanel) OR reopening a
stored one
149     // (VideoLibrary, which passes the match name). Rewrites ?video= so a refresh is
stable (D10).
150     const loadVideo = (id: string, label?: string) => {
151         setVideoId(id);
152         setVideoLabel(label ?? null);
153         window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
154     };
155
156     // Resolve the loaded video's name when we don't already have it (a deep-link ?video=
on reload, or
157     // an ingest hand-off) so the current-video label is always populated ? best-effort,
from the same
158     // GET /api/videos the library uses. A miss (not yet listed) keeps the friendly
fallback.
159     useEffect(() => {
160         if (!videoId || !videoLabel) return;
161         let alive = true;
162         listVideos()
163             .then((vs) => {
164                 const v = vs.find((x) => videoIdOf(x) === videoId);
165                 const name = v ? videoName(v) : "";
166                 if (alive && name) setVideoLabel(name);
167             })
168             .catch((err) => log("warn", "video.name_resolve_failed", { video_id: videoId,
error: String(err) }));
169         return () => {
170             alive = false;
171         };
172     }, [videoId, videoLabel]);
173
174     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
175     const onBuild = async () => {
176         if (build.status === "building") return;

```

```

177     const cid = newCorrelationId();
178     const filename = reelFilename(reelName);
179     const ids = [...sel.selected];
180     if (!videoId) {
181         log("info", "build.no_video", { cid, selected: ids.length });
182         setBuild({
183             status: "error",
184             error: t("build.noVideo"),
185         });
186         return;
187     }
188     setBuild({ status: "building" });
189     log("info", "build.start", {
190         cid,
191         video_id: videoId,
192         filename,
193         selected: ids.length,
194         total_sec: Math.round(sel.totalSec),
195         below_floor: warning?.atRisk.length ?? 0,
196     });
197     try {
198         // Pass the active UI locale so the engine localizes the reel's branding watermark
(LW1).
199         const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
?? i18n.language);
200         const url = highlightUrl(videoId, filename);
201         setBuild({ status: "done", url, filename });
202         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
203     } catch (err) {
204         // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
205         // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
206         const status = err instanceof ApiError ? err.status : undefined;
207         const e = classifyError(err);
208         setBuild({ status: "error", error: errorText(t, e) });
209         log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
210     }
211 };
212
213     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
214     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
215     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
216     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
217         const cid = newCorrelationId();
218         log("info", "query.parse", { cid, source, ...parseTrace(q) });
219
220         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
221         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
222         if (q.unavailable.length > 0) {
223             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
224         }
225
226         if (!q.recognized) {
227             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
228             return; // empty / unparseable ? leave the user's curation untouched
229         }

```

```

230
231     try {
232         const { ordered, selectedIds } = runQuery(rallies, q);
233         setOrder(ordered);
234         sel.apply(selectedIds);
235         log("info", "query.apply", {
236             cid,
237             source,
238             filters: q.filters.length,
239             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
240             limit: q.limit ?? null,
241             total: rallies.length,
242             selected: selectedIds.length,
243             selected_ids: selectedIds,
244             dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
245         });
246         if (selectedIds.length === 0) {
247             // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
248             log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
249         }
250     } catch (err) {
251         // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
252         log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
253     }
254 };
255
256 // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
257 const onToggle = (id: number) => {
258     log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
259     sel.toggle(id);
260 };
261 const onBulk = (op: "select_all" | "clear" | "invert") => {
262     const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
263     log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
264     if (op === "select_all") sel.selectAll();
265     else if (op === "clear") sel.clearAll();
266     else sel.invert();
267 };
268
269 return (
270     <main className="min-h-screen bg-paper text-ink pb-24">
271         <header className="bg-ink text-white">
272             <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
273                 <span className="text-xl font-extrabold tracking-tight">
274                     Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
275                 </span>
276                 <LanguageSwitcher />
277             </div>
278         </header>
279
280         <section className="mx-auto max-w-5xl px-6 py-10">
281             {/* Hard-blocker warning: FFmpeg absent ? indexing + reel compile can't run.
Persistent (unlike
282                 the once-per-browser wizard), shown only when the engine positively reports
it missing. */}
283             <FfmpegBanner />
284             <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-

```

```

semibold text-green">
285         {t("app.badge")}
286     </span>
287     <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
288     <p className="mt-2 max-w-2xl text-ink/70">
289         <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
290     </p>
291
292     /* Persistent "current video" label ? always visible once a video is loaded, so
the operator
293         never loses track of which match they're editing (the library/ingest panel
that showed the
294         name has unmounted by now). Name from the library, else resolved from GET
/api/videos. */}
295     {videoId && (
296         <p
297             className="mt-3 inline-flex items-center gap-2 rounded-full bg-ink/5 px-3
py-1 text-sm text-ink/70"
298             data-testid="current-video"
299         >
300             <span aria-hidden="true">?</span>
301             <Trans
302                 i18nKey="app.currentVideo"
303                 values={{ name: videoLabel ?? t("app.currentVideoUnknown") }}
304                 components={{ b: <strong className="text-ink" /> }}
305             />
306         </p>
307     )}
308
309     /* P8 (flagged): switch the loaded video between picking rallies and annotating
them. */}
310     {videoId && annotateEnabled && (
311         <div className="mt-4 inline-flex rounded-full border border-black/10 p-1 text-
sm" data-testid="view-toggle">
312             <button
313                 type="button"
314                 onClick={() => setView("pick")}
315                 aria-pressed={view === "pick"}
316                 className={"min-h-11 rounded-full px-4 " + (view === "pick" ? "bg-ink
text-white" : "text-ink/70")}
317             >
318                 {t("annotate.tabPick")}
319             </button>
320             <button
321                 type="button"
322                 onClick={() => setView("annotate")}
323                 aria-pressed={view === "annotate"}
324                 className={"min-h-11 rounded-full px-4 " + (view === "annotate" ? "bg-ink
text-white" : "text-ink/70")}
325             >
326                 {t("annotate.tab")}
327             </button>
328         </div>
329     )}
330
331     /* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
332         videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
333     {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
334
335     /* Front-half (P6 + B-P2): when no video is loaded, offer BOTH ? reopen a
stored video from
336         "Your videos" (VideoLibrary ? GET /api/videos), or add a new one

```

```

(IngestPanel ? process ?
337         load). Once a video_id exists, the existing back-half (pick ? build) takes
over. */}
338         {!videoId && wizardDone && (
339             <>
340                 <VideoLibrary onSelect={loadVideo} />
341                 <IngestPanel onLoaded={loadVideo} />
342             </>
343         )}
344
345         {/* P8 annotate view (flagged): the labeler over the loaded video. */}
346         {view === "annotate" && videoId && <AnnotatePanel key={videoId}
videoId={videoId} />}
347
348         {/* Pick-rallies view (the default + the whole app when annotate is off). */}
349         {view === "pick" && (
350             <>
351                 <ModeBanner load={loadState} videoId={videoId} />
352
353                 <div className="mt-6">
354                     <QueryBar onApply={onApplyQuery} />
355                     {/* A9: localized tap controls for users who can't type the English query
syntax. */}
356                     <TapFilters onApply={onApplyQuery} />
357                 </div>
358
359                 <div className="mt-6 flex flex-wrap gap-2 text-sm">
360                     <button type="button" onClick={() => onBulk("select_all")}
className="inline-flex min-h-11 items-center rounded-full border border-black/15 px-4
hover:border-green">{t("bulk.selectAll")}</button>
361                     <button type="button" onClick={() => onBulk("clear")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
362                     <button type="button" onClick={() => onBulk("invert")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
363                 </div>
364
365                 <div className="mt-5">
366                     <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
367                 </div>
368
369                 <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })}
/>
370             </>
371         )}
372     </section>
373
374     {/* The reel-building footer belongs to the pick view only (annotation has its own
save). */}
375     {view === "pick" && (
376         <SelectionFooter
377             count={sel.count}
378             totalSec={sel.totalSec}
379             reelName={reelName}
380             onReelNameChange={setReelName}
381             onBuild={onBuild}
382             building={build.status === "building"}
383             warning={warning}
384         />
385     )}
386 </main>
387 );
388 }

```

```

389
390 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
391   if (load.status === "demo") {
392     return (
393       <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
394         <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
395       </p>
396     );
397   }
398   if (load.status === "loading") {
399     return (
400       <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
401         <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
402       </p>
403     );
404   }
405   if (load.status === "error") {
406     return (
407       <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
408       text-red-800" data-testid="mode-banner">
409         <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
410       components={{ code: <code /> }} />
411       </p>
412     );
413   }
414   return (
415     <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
416       <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
417       <strong className="text-ink/70" /> }} />
418     </p>
419   );
420 }

```

11. assistant (2026-07-02T02:48:29.850Z)

#111 confirmed at code level: `useIngestJob.ts` holds `jobId` only inside the poll closure (lines 117-128), no localStorage/URL persistence; App.tsx seeds only from `?video=` (line 71-73). Now the panels.

12. user (2026-07-02T02:48:31.262Z)

```

1 // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2 // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3 // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4 // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5 // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6 // (done but no rallies) is shown, never silently treated as a load.
7 import { useEffect, useState } from "react";
8 import type { ChangeEvent, FormEvent } from "react";
9 import { useTranslation } from "react-i18next";
10 import { createAsset, getCapabilities } from "../api/client";
11 import type { StorageBackendInfo } from "../api/types";
12 import { useIngestJob } from "../hooks/useIngestJob";
13 import { useUpload } from "../hooks/useUpload";
14 import { classifyError, type IngestError } from "../lib/errors";
15 import { errorText } from "../lib/errorText";
16 import { log, newCorrelationId } from "../lib/log";
17 import { StatusChip, Spinner } from "../StatusChip";

```

```

18
19     type Compute = "local" | "cloud";
20     // The three storage-medium choices the picker offers. "bucket" covers s3 OR gcs (the
user pastes a
21     // full scheme'd URI); "drive" is a mounted Google Drive folder
(STUDIO_MEDIA_LIFECYCLE.md P5, D14).
22     type Medium = "local" | "bucket" | "drive";
23
24     interface Props {
25         /** Called with the engine's video_id once a submitted job finishes successfully. */
26         onLoaded: (videoId: string) => void;
27     }
28
29     export function IngestPanel({ onLoaded }: Props) {
30         const { t, i18n } = useTranslation();
31         const [uri, setUri] = useState("");
32         const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
33         // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
34         const locale = i18n.resolvedLanguage ?? i18n.language;
35
36         // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
37         // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
38         // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it
can run.
39         const [cloudAvailable, setCloudAvailable] = useState(false);
40         const [compute, setCompute] = useState<Compute>("local");
41         const [cloudAck, setCloudAck] = useState(false);
42         // Storage-medium picker (P5): which media THIS box can store into
(capabilities.storage.backends,
43         // usable-only). Absent/local-only ? no picker (today's flow unchanged).
44         const [backends, setBackends] = useState<StorageBackendInfo[]>([]);
45         const [medium, setMedium] = useState<Medium>("local");
46         const [mediumUri, setMediumUri] = useState("");
47         const [storing, setStoring] = useState(false);
48         const [storeError, setStoreError] = useState<IngestError | null>(null);
49         useEffect(() => {
50             let alive = true;
51             getCapabilities()
52                 .then((c) => {
53                     if (!alive) return;
54                     setCloudAvailable(c.deployment?.cloud_available === true);
55                     setBackends(c.storage?.backends ?? []);
56                 })
57                 .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
58             return () => {
59                 alive = false;
60             };
61         }, []);
62         // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
63         const sendCompute = cloudAvailable ? compute : undefined;
64         // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
65         // don't dispatch a paid job without consent).
66         const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;
67
68         // Which medium segments to offer: local always; "bucket" if s3 OR gcs is usable;
"drive" if a Drive
69         // mount is usable. The picker only appears when there's a REAL choice (a non-local
usable backend).
70         const canUse = (id: string) => backends.some((b) => b.id === id && b.usable);

```

```

71     const canBucket = canUse("s3") || canUse("gcs");
72     const canDrive = canUse("gdrive");
73     const mediumOptions: Medium[] = ["local", ...(canBucket ? ["bucket" as const] : []),
...(canDrive ? ["drive" as const] : [])];
74     const showMediumPicker = mediumOptions.length > 1;
75     const needsMediumUri = medium !== "local";
76     // A non-local medium needs a destination URI before anything can be stored/submitted.
77     const mediumBlocked = needsMediumUri && mediumUri.trim() === "";
78
79     // Store the obtained source (an uploaded local path, or a pasted at-rest URI) INTO
the chosen medium,
80     // then process the durable copy. A failed store is loud (surfaced like every other
ingest error).
81     const storeThenProcess = async (src: { uploadedPath?: string; sourceUri?: string }) =>
{
82         const cid = newCorrelationId();
83         setStoreError(null);
84         setStoring(true);
85         try {
86             const asset = await createAsset({ ...src, mediumUri: mediumUri.trim() }, cid);
87             log("info", "asset.stored", { cid, video_id: asset.video_id, medium: asset.medium,
copied: asset.copied });
88             submit(asset.stored_uri, locale, sendCompute); // index the stored copy
89         } catch (err) {
90             setStoreError(classifyError(err));
91             log("error", "asset.store_failed", { cid, error: String(err) });
92         } finally {
93             setStoring(false);
94         }
95     };
96
97     // On upload completion: local medium ? process the upload directly (today's flow); a
non-local
98     // medium ? store a durable copy into it first, then process. The arrow is recreated
each render
99     // (useUpload reads onCompleteRef.current), so `medium`/`mediumUri`/`sendCompute` are
always fresh.
100    const upload = useUpload((path) => {
101        if (medium === "local") submit(path, locale, sendCompute);
102        else void storeThenProcess({ uploadedPath: path });
103    });
104    const busy = ingestBusy || upload.busy || storing;
105    // Disable the submit surfaces while busy, awaiting the cloud cost ack, or missing a
medium URI.
106    const inputsDisabled = busy || cloudBlocked || mediumBlocked;
107
108    const onSubmitUri = (e: FormEvent) => {
109        e.preventDefault();
110        if (inputsDisabled) return;
111        // The URI form is the SOURCE only in local mode (a bucket URL / host path to index
directly). For
112        // a non-local medium the source is the upload; the text box below is the
DESTINATION, not this.
113        submit(uri, locale, sendCompute);
114    };
115    const onFile = (e: ChangeEvent<HTMLInputElement>) => {
116        const file = e.target.files?.[0];
117        // Symmetric with onSubmitUri: never start while blocked (the disabled input already
prevents this).
118        if (file && !inputsDisabled) upload.start(file);
119        e.target.value = ""; // let re-picking the same file fire change again
120    };
121    const cancelAll = () => {
122        upload.cancel();
123        cancelIngest();

```

```

124     };
125     const resetAll = () => {
126         upload.reset();
127         resetIngest();
128         setStoreError(null);
129     };
130
131     // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
132     // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
133     // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
134     const errorState =
135         upload.phase.status === "error"
136         ? upload.phase.error
137         : storeError ?? (phase.status === "error" ? phase.error : null);
138     const errorMsg = errorState ? errorText(t, errorState) : null;
139
140     return (
141         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
142             <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
143             <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
144
145             {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
146                 LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
147             {cloudAvailable && (
148                 <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
149                     <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
150                         {t("ingest.compute.legend")}
151                     </legend>
152                     <div className="mt-1 flex flex-wrap gap-2">
153                         {[["local", "cloud"] as const].map((opt) => (
154                             <button
155                                 key={opt}
156                                 type="button"
157                                 aria-pressed={compute === opt}
158                                 disabled={busy}
159                                 onClick={() => setCompute(opt)}
160                                 className={
161                                     "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
162                                     (compute === opt
163                                         ? "border-green bg-green/10 text-ink"
164                                         : "border-black/15 text-ink/70 hover:border-green")
165                                 }
166                             >
167                                 {t(`ingest.compute.${opt}`)}
168                             </button>
169                         )]}
170                     </div>
171                     {compute === "cloud" && (
172                         <div className="mt-3" data-testid="compute-cost">
173                             <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
174                             <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
175                                 <input
176                                     type="checkbox"
177                                     checked={cloudAck}
178                                     disabled={busy}
179                                     onChange={(e) => setCloudAck(e.target.checked)}

```

```

180         className="h-4 w-4"
181     />
182     {t("ingest.compute.ack")}
183 </label>
184     {cloudBlocked && (
185         <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
186             {t("ingest.compute.ackRequired")}
187         </p>
188     )}
189 </div>
190 )}
191 </fieldset>
192 )}
193
194     {/* MEDIUM PICKER (P5, D14): where to STORE this video. Rendered only from the
media this box can
195         actually use (capabilities.storage.backends, usable-only), and only when
there's a real
196         choice beyond local. A non-local medium reveals a destination-URI field; the
video is stored
197         there (a durable, MD5-keyed copy ? free-space checked) and then indexed. Self-
host posture. */}
198     {showMediumPicker && (
199         <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="medium-picker">
200             <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
201                 {t("ingest.medium.legend")}
202             </legend>
203             <div className="mt-1 flex flex-wrap gap-2">
204                 {mediumOptions.map((opt) => (
205                     <button
206                         key={opt}
207                         type="button"
208                         aria-pressed={medium === opt}
209                         disabled={busy}
210                         onClick={() => {
211                             setMedium(opt);
212                             setStoreError(null);
213                         }}
214                         data-testid={`medium-${opt}`}
215                         className={
216                             "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
217                             (medium === opt
218                                 ? "border-green bg-green/10 text-ink"
219                                 : "border-black/15 text-ink/70 hover:border-green")
220                         }
221                     >
222                         {t(`storage.${opt}`)}
223                     </button>
224                 )}
225             </div>
226             {needsMediumUri && (
227                 <div className="mt-3" data-testid="medium-dest">
228                     <input
229                         type="text"
230                         value={mediumUri}
231                         onChange={(e) => setMediumUri(e.target.value)}
232                         placeholder={medium === "drive" ? t("ingest.medium.drivePlaceholder") :
t("ingest.medium.bucketPlaceholder")}
233                         aria-label={t("ingest.medium.destAria")}
234                         disabled={busy}
235                         className="min-h-11 w-full max-w-md rounded-xl border border-black/15

```

```

bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
236         />
237         <p className="mt-2 text-xs text-ink/50">{t("ingest.medium.destHint")}</p>
238         {mediumBlocked && (
239             <p className="mt-1 text-xs text-amber-700" data-testid="medium-
required">
240                 {t("ingest.medium.destRequired")}
241             </p>
242         )}
243     </div>
244 )}
245 </fieldset>
246 )}
247
248 {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
249 <div className="mt-4">
250     <label
251         className={
252             // focus-within makes the visible button show a focus ring when the (sr-
only) input is
253             // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
254             "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
255             (inputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
256         }
257     >
258         {t("ingest.chooseFile")}
259         <input
260             type="file"
261             accept=".mp4,.mov,video/mp4,video/quicktime"
262             aria-label={t("ingest.uploadAria")}
263             disabled={inputsDisabled}
264             onChange={onFile}
265             className="sr-only"
266         />
267     </label>
268     <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
269 </div>
270
271 {/* SECONDARY: in LOCAL mode, point at a cloud bucket / host path to index
directly. Hidden for a
272     non-local medium, where the destination field above owns the URI box and the
upload is the
273     source (avoids two conflicting URI inputs). */}
274 {medium === "local" && (
275     <div className="mt-4 border-t border-black/5 pt-4">
276         <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
277         <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
278             <input
279                 type="text"
280                 value={uri}
281                 onChange={(e) => setUri(e.target.value)}
282                 placeholder={t("ingest.uriPlaceholder")}
283                 aria-label={t("ingest.uriAriaLabel")}
284                 disabled={inputsDisabled}
285                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
286             />
287             <button
288                 type="submit"
289                 disabled={inputsDisabled}
290                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-

```

```

white disabled:opacity-40"
291         >
292         {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
293     </button>
294 </form>
295     <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
296 </div>
297     )}
298
299     {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
300         (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
301     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
302         {upload.phase.status === "uploading" && (
303             <div data-testid="upload-progress">
304                 {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
305                     % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
306                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
307                 <div className="mt-2 flex max-w-sm items-center gap-2">
308                     <div
309                         className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
310                         role="progressbar"
311                         aria-label={t("ingest.uploadProgress")}
312                         aria-valuenow={upload.phase.pct}
313                         aria-valuemin={0}
314                         aria-valuemax={100}
315                     >
316                         <div className="h-full bg-green transition-all" style={{ width:
`$${upload.phase.pct}%` }} />
317                     </div>
318                     <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
319                         {upload.phase.pct}%
320                     </span>
321                 </div>
322                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
323                 <button
324                     type="button"
325                     onClick={cancelAll}
326                     className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
327                 >
328                     {t("ingest.cancel")}
329                 </button>
330             </div>
331         )}
332         {storing && (
333             <div data-testid="ingest-storing">
334                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.medium.storing")} />
335             </div>
336         )}
337         {ingestBusy && (
338             <div>
339                 <StatusChip
340                     tone="info"
341                     icon={<Spinner />}
342                     label={
343                         t("ingest.working") +
344                         (phase.status === "polling" && phase.progress
? ` · ${t("ingest.progress", { progress: phase.progress })}`
345

```

```

346         : "")
347     }
348     />
349     /* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
350     <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
351     <button
352         type="button"
353         onClick={cancelAll}
354         className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
355     >
356         {t("ingest.cancel")}
357     </button>
358 </div>
359 )}
360 {errorMsg && (
361     <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
362         <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
363         <p className="mt-2">{errorMsg}</p>
364         <button
365             type="button"
366             onClick={resetAll}
367             className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
368         >
369             {t("ingest.retry")}
370         </button>
371     </div>
372 )}
373 </div>
374 </section>
375 );
376 }
377

```

13. user (2026-07-02T02:48:32.029Z)

```

1      // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2      // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3      // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4      // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6      import { useTranslation } from "react-i18next";
7      import { StatusChip, Spinner } from "../StatusChip";
8
9      export type BuildState =
10         | { status: "idle" }
11         | { status: "building" }
12         | { status: "done"; url: string; filename: string }
13         | { status: "error"; error: string };
14
15      interface Props {
16         build: BuildState;
17         onDismiss: () => void;
18     }
19
20      export function CompileResult({ build, onDismiss }: Props) {
21         const { t } = useTranslation();
22         if (build.status === "idle") return null;

```

```

23
24      // Build state (A8): a visible "stitching?" card with an icon+text chip + reassurance,
so the user
25      // isn't staring at a frozen footer wondering whether anything is happening.
26      if (build.status === "building") {
27          return (
28              <section data-testid="compile-result" className="mt-6 rounded-xl border border-
black/10 bg-paper p-4" aria-live="polite">
29                  <StatusChip tone="info" icon={<Spinner />} label={t("result.building")} />
30                  <p className="mt-2 text-sm text-ink/60">{t("result.buildingReassure")}</p>
31              </section>
32          );
33      }
34
35      const isError = build.status === "error";
36      return (
37          <section
38              data-testid="compile-result"
39              aria-live="polite"
40              className={"mt-6 rounded-xl border p-4 " + (isError ? "border-red-200 bg-red-50" :
"border-green/30 bg-green/5")}
41          >
42              <div className="flex items-start justify-between gap-3">
43                  {/* Status as icon+text (never icon-only): ? ready / ? failed (A8). */}
44                  <h2 className="flex items-center gap-2 font-semibold text-ink">
45                      <span aria-hidden="true" className={isError ? "text-red-700" : "text-
green"}>{isError ? "?" : ""}</span>
46                      {build.status === "done" ? t("result.ready") : t("result.failed")}
47                  </h2>
48                  <button
49                      type="button"
50                      onClick={onDismiss}
51                      aria-label={t("result.dismiss")}
52                      className="flex h-11 w-11 shrink-0 items-center justify-center rounded-full
text-ink/50 hover:text-ink"
53                  >
54                      ?
55                  </button>
56              </div>
57
58              {build.status === "done" ? (
59                  <div className="mt-3">
60                      {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
61                      <video
62                          data-testid="reel-video"
63                          src={build.url}
64                          controls
65                          playsInline
66                          className="w-full max-w-xl rounded-lg bg-black"
67                      />
68                      <div className="mt-3 flex flex-wrap items-center gap-2">
69                          <a
70                              href={build.url}
71                              download={build.filename}
72                              className="inline-flex min-h-11 items-center rounded-full bg-green px-4
py-2 text-sm font-semibold text-[#04140D]"
73                          >
74                              {t("result.download", { filename: build.filename })}
75                          </a>
76                      {/* A2: localized WhatsApp share. Opens the user's own WhatsApp with a pre-
filled message
77                          (wa.me can't attach the local file, so the user adds the just-
downloaded reel). */}
78                      <a

```

```

79         href={`https://wa.me/?text=${encodeURIComponent(t("result.shareText"))}`}
80         target="_blank"
81         rel="noopener noreferrer"
82         data-testid="whatsapp-share"
83         className="inline-flex min-h-11 items-center gap-1.5 rounded-full border
border-green/50 px-4 py-2 text-sm font-semibold text-green hover:bg-green/5"
84     >
85         <span aria-hidden="true">?</span>{t("result.share")}
86     </a>
87 </div>
88 </div>
89 ) : (
90     <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
91         {build.error}
92     </p>
93 )}
94 </section>
95 );
96 }
97

```

14. user (2026-07-02T02:48:43.823Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable

```

```

33     if (err.body) {
34         const detail = parseDetail(err.body);
35         if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36     }
37     // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38     // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39     // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40     if (err.status >= 500) return { code: "networkError" };
41     return { code: "engine", detail: `HTTP ${err.status}` };
42 }
43 return { code: "engine", detail: String(err) };
44 }
45

```

15. user (2026-07-02T02:48:44.913Z)

```

1     // Resilient readers for a GET /api/videos row (VideoSummary). Defense-in-depth against
engine drift:
2     // the engine SHOULD return the contract fields (`video_id`, a friendly `match_name`,
`fps`,
3     // `resolution`), but if a build regresses to the raw DB shape (`id`, `filepath`, no
name), the library
4     // + the persistent current-video label still work ? they read through these helpers,
never the raw
5     // field. (The bug this guards against actually shipped once: the real engine returned
`id`/no-name
6     // while the e2e mock faked both, so a "green" UI was broken live.)
7     import type { VideoSummary } from "../api/types";
8
9     /** The MD5 join key ? falls back to the raw `id` column if `video_id` is absent. */
10    export function videoId(v: VideoSummary): string {
11        return v.video_id ?? v.id ?? "";
12    }
13
14    /** Filename stem of a path/URI: `~/My Match 2026.mp4` ? `My Match 2026`. Handles `~/`
and `\\`,
15    * a trailing slash, and a no-extension name; returns "" for empty input. */
16    export function basenameStem(path: string | null | undefined): string {
17        const base =
18            String(path ?? "")
19                .replace(/[/\\]+$/, "")
20                .split(/[/\\]/)
21                .pop() ?? "";
22        const dot = base.lastIndexOf(".");
23        return dot > 0 ? base.slice(0, dot) : base;
24    }
25
26    /** A human name ? the engine's `match_name`, else derived from the stored path/URI,
else "" (the
27    * caller applies its own localized "unnamed"/"your video" fallback). */
28    export function videoName(v: VideoSummary): string {
29        return v.match_name || basenameStem(v.local_path ?? v.filepath);
30    }
31
32    /** A compact spec line, e.g. `1920x1080 · 30fps` (either part optional; "" when neither
is known). */
33    export function videoSpec(v: VideoSummary): string {
34        const parts: string[] = [];
35        if (v.resolution) parts.push(v.resolution);
36        if (typeof v.fps === "number" && v.fps > 0) parts.push(`${Math.round(v.fps)}fps`);
37        return parts.join(" · ");

```

```
38     }
39
```

16. user (2026-07-02T02:48:45.981Z)

```
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\App.tsx:258:    log("debug",
"rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\i18n\catalog-invariants.test.ts:30://
(1) Internal-token / debug-tag leaks ? EVERY locale. The engine's internal `min_shot_count`
config
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\i18n\catalog-invariants.test.ts:31://
name leaked into footer.floorSomeRest (#77, kn even wrapped it in <code>). No developer/debug
tag
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\i18n\catalog-
invariants.test.ts:37:describe("catalog content ? no internal token / debug-tag leaks (every
locale)", () => {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\i18n\catalog-invariants.test.ts:39:
const DEBUG_TAG = /\<\/?(?:kbd|samp|tt|var)\b/i;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\i18n\catalog-invariants.test.ts:45:
.filter(([k, v]) => TOKEN.test(v) || DEBUG_TAG.test(v) || (CODE_TAG.test(v) && !codeAllowed(k)))
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\log.ts:6:export type LogLevel =
"debug" | "info" | "warn" | "error";
```

17. user (2026-07-02T02:48:47.183Z)

No matches found

18. user (2026-07-02T02:49:07.468Z)

```
1      // The ONE place a classified error (lib/errors.ts) becomes user-facing text. Both
surfaces use it:
2      // the ingest flow (IngestPanel) and the build flow (App.onBuild ? CompileResult), so a
5xx / engine-
3      // down failure reads the same friendly way everywhere instead of a raw "HTTP 500" /
fetch message.
4      //
5      // `engine` carries the engine's OWN message (the most specific) ? shown verbatim, never
translated.
6      // Every other code maps to a friendly localized line in the shared `errors.*`
namespace. Kept out of
7      // errors.ts so that module stays i18n-free and unit-testable without a translation
context.
8      import type { TFunction } from "i18next";
9      import type { IngestError } from "../errors";
10
11      export function errorText(t: TFunction, e: IngestError): string {
12          if (e.code === "engine") return e.detail ?? t("errors.engineGeneric");
13          return t(`errors.${e.code}`);
14      }
15
```

19. user (2026-07-02T02:49:07.905Z)

```
1      // Types mirror the engine's verified contract (badminton-highlight-indexer).
2      // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3      // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5      export interface RallyMetadata {
6          /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7          shot_count?: number;
8          /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9          shot_count_confidence?: string;
10         detector?: string;
```

```

11     [key: string]: unknown;
12 }
13
14 export interface RallySegment {
15     /** The integer the UI selects and passes to /api/compile as segment_ids. */
16     id: number;
17     start_time: number;
18     end_time: number;
19     duration: number;
20     /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 /** One row of the indexed-video library. GET /api/videos [backend/main.py:447] returns
the most
34 * recent first so the console survives a reload. Every field but `video_id` is
optional/defensive. */
35 export interface VideoSummary {
36     /** The MD5 join key (the contract field). Optional defensively ? an older/regressed
engine may
37 * send only the raw `id`; read it through `lib/video.ts::videoId`, never directly.
*/
38     video_id?: string;
39     /** Raw DB fields an engine may send instead of the enriched contract (resilience
fallbacks). */
40     id?: string;
41     filepath?: string | null;
42     match_name?: string | null;
43     local_path?: string | null;
44     fps?: number | null;
45     resolution?: string | null;
46     created_at?: string | null;
47 }
48
49 export interface VideosResponse {
50     videos: VideoSummary[];
51 }
52
53 /** POST /api/annotations ? persist finished rally labels for a video as the annotator
CSV (P8).
54 * The engine writes the CSV verbatim (byte-compatible with the pipeline), keyed by the
file MD5. */
55 export interface AnnotationsResponse {
56     status: string;
57     num_rows?: number;
58     csv_uri?: string;
59     message?: string;
60 }
61
62 export interface CompileResponse {
63     status: string;
64     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
65     filepath: string;
66 }
67

```

```

68     /** The terminal RESULT of an async reel compile (#329): the job's `result` payload,
surfaced by
69     * GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict
("success" |
70     * "failed"); a "failed" result carries a `message`. The SPA builds the download URL
with
71     * highlightUrl() (split-origin-safe), but the engine also returns a same-origin
`download_url`. */
72     export interface CompileResult {
73         status: string;
74         filepath?: string;
75         download_url?: string;
76         message?: string;
77         video_id?: string;
78     }
79
80     export interface EngineConfig {
81         stitching?: {
82             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
83             min_shot_count?: number;
84             [key: string]: unknown;
85         };
86         [key: string]: unknown;
87     }
88
89     /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
90     * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini
key value).
91     * Every field optional/defensive: a missing field degrades the wizard gracefully. */
92     export interface EngineCapabilities {
93         api_version?: number;
94         engine_version?: string;
95         segmenters?: string[];
96         default_segmenter?: string;
97         ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
98         gpu?: { nvidia_smi_present?: boolean };
99         /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100         gemini_key_present?: boolean;
101         wasb_weights_present?: boolean;
102         /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
103         * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
104         deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
105         /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
106         * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
107         * ? the SPA degrades to local-only (no picker). */
108         storage?: StorageCapabilities;
109         [key: string]: unknown;
110     }
111
112     /** One storable medium reported by GET /api/capabilities (P3). Presence/usable bools
only ? no
113     * secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
114     export interface StorageBackendInfo {
115         id: string;
116         kind: string;
117         label: string;
118         usable: boolean;

```

```

119     /** Free space in whole GB, or `null` when unknowable (a cloud bucket). */
120     free_gb?: number | null;
121 }
122
123 export interface StorageCapabilities {
124     default_medium?: string;
125     backends?: StorageBackendInfo[];
126 }
127
128 /** POST /api/assets ? 201 (P4). The video was stored into the chosen medium and
registered as a
129 * library asset keyed by the whole-file MD5 (`video_id`). `stored_uri` is the durable
location to
130 * feed into POST /api/process (a bucket/Drive URI, or a local path for "this
computer"). */
131 export interface AssetResponse {
132     video_id: string;
133     /** The medium backend the asset lives in (local | s3 | gcs | gdrive). */
134     medium: string;
135     /** The durable, MD5-keyed location ? pass as video_path to processVideo(). */
136     stored_uri: string;
137     /** True when a durable copy was made (uploaded ? remote medium); false when
registered in place. */
138     copied: boolean;
139     sport?: string | null;
140     status?: string;
141 }
142
143 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
144 export interface ProcessResponse {
145     job_id: string;
146     /** Always "queued" at submit. */
147     status: string;
148     /** Relative poll path, e.g. "/api/jobs/<id>". */
149     poll: string;
150 }
151
152 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
153 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
154 export interface UploadInitResponse {
155     upload_id: string;
156     /** Per-part ceiling in MB ? slice the file into parts this big. */
157     max_part_mb: number;
158     next_part: string;
159 }
160 export interface UploadPartResponse {
161     upload_id: string;
162     received_parts: number;
163     received_bytes: number;
164 }
165 export interface UploadCompleteResponse {
166     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
167     path: string;
168     video_id: string;
169     size_bytes: number;
170 }
171
172 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
173 * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
174 * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
shaped

```

```

175     * FAILURE ? the UI must not treat it as a load. */
176     export interface JobStatus {
177         job_id: string;
178         status: "queued" | "running" | "done" | "failed";
179         /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
180         progress?: string | null;
181         /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
182         video_id?: string | null;
183         /** Execution error (worker crash / unexpected) when status === "failed". */
184         error?: string | null;
185         result?: { status?: string; video_id?: string; message?: string } | null;
186     }
187

```

20. user (2026-07-02T02:49:08.626Z)

```

1     // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2     // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3     // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4     // keystone that lets them point the tool at it from the browser.
5     //
6     // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7     // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8     // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9     // orchestrator best-effort DELETES the server-side partial.
10    import { useCallback, useEffect, useRef, useState } from "react";
11    import { uploadFile } from "../api/client";
12    import { classifyError, type IngestError } from "../lib/errors";
13    import { log, newCorrelationId } from "../lib/log";
14
15    export type UploadPhase =
16        | { status: "idle" }
17        | { status: "uploading"; pct: number; filename: string }
18        | { status: "error"; error: IngestError };
19
20    export function useUpload(onComplete: (path: string) => void) {
21        const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22        const ctrlRef = useRef<AbortController | null>(null);
23        // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24        const onCompleteRef = useRef(onComplete);
25        onCompleteRef.current = onComplete;
26
27        const stop = useCallback(() => {
28            ctrlRef.current?.abort();
29            ctrlRef.current = null;
30        }, []);
31
32        // Abort any in-flight upload on unmount.
33        useEffect(() => stop, [stop]);
34
35        const start = useCallback(
36            (file: File) => {
37                // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38                if (!file || file.size === 0) {
39                    setPhase({ status: "error", error: { code: "invalidUri" } });

```

```

40         return;
41     }
42
43     stop(); // supersede any prior upload
44     const ctrl = new AbortController();
45     ctrlRef.current = ctrl;
46     const cid = newCorrelationId();
47     setPhase({ status: "uploading", pct: 0, filename: file.name });
48     log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50     void (async () => {
51         try {
52             const res = await uploadFile(file, {
53                 cid,
54                 signal: ctrl.signal,
55                 onProgress: ({ sentBytes, totalBytes }) => {
56                     if (ctrl.signal.aborted) return;
57                     const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                     setPhase({ status: "uploading", pct, filename: file.name });
59                 },
60             });

```

21. user (2026-07-02T02:49:32.642Z)

```

locales/en/common.json:95:     "uploadingStatus": "Uploading?",
95:     "uploadingStatus": "Uploading?",

```

22. user (2026-07-02T02:49:33.880Z)

```

60         });
61         if (ctrl.signal.aborted) return;
62         setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63         onCompleteRef.current(res.path);
64     } catch (err) {
65         // User-cancelled (abort) is silent, not an error.
66         if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67         setPhase({ status: "error", error: classifyError(err) });
68         log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69     }
70     })();
71     },
72     [stop],
73 );
74
75     const cancel = useCallback(() => {
76         stop();
77         setPhase({ status: "idle" });
78         log("info", "upload.cancel", {});
79     }, [stop]);
80
81     const reset = useCallback(() => {
82         stop();
83         setPhase({ status: "idle" });
84     }, [stop]);
85
86     const busy = phase.status === "uploading";
87     return { phase, start, cancel, reset, busy };
88 }
89

```

23. user (2026-07-02T02:49:34.853Z)

```
10:import { videoId, videoName, videoSpec } from "../lib/video";
79:      const meta = [videoSpec(v), v.created_at].filter(Boolean).join(" · ");
```

24. user (2026-07-02T02:49:46.519Z)

```
deploy:
DEPLOY_ALPHA.md
GPU_VM_SERVING.md
cloudflared.config.example.yml
droplet
stage_from_bucket.sh

docs:
ANNOTATE_AND_TRAIN.md
DRY_RUN_CUJ.md
LOCAL_APPLIANCE_ARCHITECTURE.md
MULTILINGUAL_EXPANSION.md
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md
STUDIO_MEDIA_LIFECYCLE.md
archives
deploy/cloudflared.config.example.yml
deploy/DEPLOY_ALPHA.md
deploy/droplet/engine.env.example
deploy/droplet/khelsutra-engine.service
deploy/droplet/README.md
deploy/GPU_VM_SERVING.md
deploy/stage_from_bucket.sh
docs/DRY_RUN_CUJ.md
docs/LOCAL_APPLIANCE_ARCHITECTURE.md
docs/PORTABILITY.md
```

25. user (2026-07-02T02:49:53.306Z)

Deploy KhelSutra Studio for alpha ? a shareable page, no SSH

```
> Status: `LIVE` (deploy live-verified on the droplet 2026-06-22 ?
`deploy/droplet/README.md`; only a rotated Gemini key + a browser OTP login remain to exercise
the upload?reel CUJ) · Topology: split ? SPA on Cloudflare Pages + engine on an `api.`
cloudflared tunnel, Cloudflare Access over both (the engine's `HOSTING_PLAN.md` S2 alpha plan).
· Cost: ~$0/mo standing (Pages + Tunnel + Access are free ?50 users) + Gemini per-run
(~$0.05/match).
```

This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a **page** you can share with alpha testers ? they open `app.khelsutra.guru`, sign in (email allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.

```
> Outgrown the droplet? To move the live engine onto a GCP L4 GPU VM (more CPU/RAM for
ffmpeg
> + optional WASB GPU detection) and cut the tunnel over to it, see
GPU_VM_SERVING.md (GPU_VM_SERVING.md)
> ? same tunnel + Access, no DNS change, droplet kept as fallback (~$500?600/mo while the VM
runs).
```

0. The shape

```
...
tester browser
? https
```

```

?
Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH hostnames
??? app.khelsutra.guru ? Cloudflare Pages (the web/ SPA; built with VITE_API_BASE)
??? api.khelsutra.guru ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on YOUR box
                        ? FastAPI engine
                        ? (Gemini segmenter; GPU
optional)
                        ? async jobs, SQLite,
output/ on disk
```

```

```

- **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`. The
box's home/public IP is never exposed.
- **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies the
`Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to the
tunnel can't spoof identity.
- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the SPA
is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks CORS
to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-
driven).

> **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA *and* the
API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).

```

## 1. Prerequisites

```

- A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU; fine for
alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed; the
`badminton-highlight-indexer` engine checked out.
- A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing site
there).
- `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-
one/connections/connect-networks/downloads/`).
- A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks to
Google; this is a paid + cloud action).

```

## 2. Engine (the `api.` side)

### 2.1 Install (with the auth extra)

```

```bash
cd badminton-highlight-indexer
pip install -e .[auth] # PyJWT[crypto] ? REQUIRED when edge auth is on, else every
authenticated request is rejected 401
```

```

### 2.2 The safe-exposed config

```

Create / edit `config.json` on the box. **Where it must live:** the engine reads
`$RALLY_APP_HOME/config.json` if `RALLY_APP_HOME` is set, else `./config.json` relative to *the
directory you launch the server from* ? **not** necessarily the repo root. If the engine reads a
different (or absent) file it silently falls back to all-defaults, which means **edge auth is
OFF**. To make writer and reader agree by construction, write it with `python -m backend.main
--setup` (it targets the exact path the server reads) and edit *that* file:
```jsonc
{
  "security": {
    "edge_auth_enabled": true, // turn the CF-Access JWT verify ON
    "allowed_video_root": "/srv/khelsutra/incoming", // the path-jail ? REQUIRED when exposed
    "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER allowed_video_root (see

```

```

$7)
  "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
  "cf_access_aud": "<the CF Access application AUD>" // filled in $4
},
====
5: This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a **page
you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email allowlist),
point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
18:   ??? app.khelsutra.guru ? Cloudflare Pages          (the web/ SPA; built with
VITE_API_BASE)
29:> **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA *and*
the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
69: export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru" # lock CORS to the SPA origin
96: 3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.
105: 2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to
the *same* application (one app ? one shared session cookie, so a tester signs in once and both
the SPA and its API calls are authorized).
130: - Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA
loads and `/api/health` returns 200 (same session cookie). ?
132: **The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ? **upload a
video from their device** (the SPA chunked-upload lands it in `upload_dir`, inside the jail) ?
**Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the
page. *(Pasting a `gs://` URI does NOT work on this box ? see $7: the `allowed_video_root` jail
rejects every non-local URI. Browser upload is the alpha path.)*
154: - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access)
consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE_API_BASE` (#64).
Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative) + the engine
`HOSTING_PLAN.md` $2.

```

26. user (2026-07-02T02:49:54.197Z)

Next steps (live) ? KhelSutra brand / site

Prioritized "what to do next" for the **`khelsutra` product repo**. Refresh at each checkpoint.

```

**Where we are (updated 2026-07-01):** `khelsutra.guru` is **LIVE and fully shipped** ? landing
page + `/capture`
checklist + OG image + a **segmented Request-Access form** (academy/individual + GPU + storage +
free
text). On **Cloudflare** (Worker `hidden-king-khelsutra` + D1 `khelsutra-access`): the form
persists to
D1 **and emails a summary to the owner on every registration** (Email Routing send binding ?
verified
live). **Git auto-deploy is connected** (Workers Builds, root `site/`): merge to `master` ?
deploys;
**CI gates every PR** ? three jobs: the full site `npm test` (Worker + VM host + assets + UX),
the
`web/` SPA build + coverage floor, and a real-browser layout check. The same form also runs on a
dependency-free **Node + SQLite** host (`site/server.js`), **live + browsable on the DO droplet
?
http://168.144.126.176:8787** (systemd `khelsutra-site`). Shipped across ~90 merged PRs (the
form,
6-language localization, the per-rally MVP, droplet portability). Docs:
`docs/REQUEST_ACCESS_FORM.md`, `docs/PORTABILITY.md`.

```

Next product threads (owner ideas, 2026-06-20)

```

- **? DONE ? Friendly & Multilingual (ARCHIVED 2026-06-22; human gates cleared):** the product
is
  non-tech-friendly and **fully localized to all 6 languages** (en/kn/hi/es/da/id) across both
surfaces
  (`web/` SPA + `site/` marketing) ? including the home page (hero, all sections, the `#join`
access form,

```

footer), the **Privacy Policy** page, **per-page** `<title>s`, and **es/da/id academy flyers** (with localized SPA screenshots). The responsive header is fixed and a **headless-Chromium CI suite** guards layout, cross-locale render (live DOM == catalog), and **accessibility** (axe-core WCAG 2.1 AA). Full record: `docs/archives/FRIENDLY_AND_MULTILINGUAL.md`. Post-archive PRs: #86 (own-model) · #87 (es/da/id site) · #105 (form+footer + render guard) · #106 (Privacy page) · #107 (a11y fixes + axe guard + page titles) · #109 (flyers). **Owner gates CLEARED:** native-speaker review of the AI-drafted catalogs ? + real-device font-glyph eyeball ?. **Remaining (small/owner):**

- **Delta native-review** of the strings ADDED after the owner's sign-off (#105/#106/#107/#109 ? the form/footer/privacy/title/flyer copy is AI-drafted): a small top-up review before a wide public launch.
- **B7 (deferred, not a v1 blocker):** engine backend customer-facing l10n (report `customer_`` + rejection reasons) ? cross-repo Python (`badminton-highlight-indexer``), owner-gated **D-BACKEND-DEPTH**.
- **Marketing video on the site** ? embed a short walkthrough of the record ? index ? reel process, themed for **serious enthusiasts**, on the landing page. ? **A real-UI CUJ screencast** now exists (Playwright recording of the per-rally SPA driven against real GPU-run data ? `promo_cuj.mp4`/.gif`` in `gs://khelesutra-rally-corpus/promo/``). Decide: use it as the marketing clip (? shows real footage + a dev "P5" badge ? clean up before public embed), and/or produce a polished Grok/Gemini cut.
- **Interactive per-rally selection** (big product feature) ? ? **MVP / M0 COMPLETE** (the full pick ? query ? select ? compile ? download loop on existing engine endpoints). Lives in the alpha SPA (`web/``). **Tracked design + P-tracker (source of truth):** `docs/PER_RALLY_SELECTION.md`` (IN PROGRESS; owner-approved, D8?D11 locked). **Done:** P0 design (#11), P1 scaffold (#14), P2 grid+hook (#21), P3

====

f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelesutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
a1d9e41 docs(decision): ratify P2-P5 medium-track decisions (D14) ? track is turnkey (#135)
180a747 docs(tracker): accuracy pass ? reconcile the project doc with shipped state (#134)
dfc643f docs(tracker): annotation proxy landed (D4, engine #406) (#133)
6a6ac47 docs(tracker): P9 done ? collector bridge + write-safety (labeled_window deferred) (#132)
7487a9b docs(tracker): P9 ? import_annotations + maturity gate landed (collector #52) (#131)
ac3597a docs(tracker): P9 in progress ? MD5 join key landed (collector #51) (#130)
bb35e5e docs(tracker): P8 complete ? engine annotate endpoints landed (P8a) (#129)
cc06c32 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
9e9df72 feat(web): keep the video name visible after open (persistent current-video label) (#127)
926b53a feat(web): vendor the rally-annotator annotation core (P8 groundwork) (#126)

27. user (2026-07-02T02:50:16.134Z)

/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelesutra-guru/memory/compute-stack-gcp-only.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelesutra-guru/memory/current-state-archive.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelesutra-guru/memory/current-state.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelesutra-guru/memory/golden-corpus-

```

grew-9-to-15.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/khelsutra-site-
droplet-parity.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/MEMORY.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/packaging-app-
plan.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/session-handoff.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/working-root-
khelsutra-guru.md
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/MEMORY.md:63:-
[khelsutra site droplet parity](khelsutra-site-droplet-parity.md) ? **RETIRED 2026-06-25.** DO
droplet `claude-do-rally-test` (dual-purpose site mirror + alpha engine) kept drifting from CF ?
owner retired it: data archived to GCS+local, droplet deleted ($0), monitor removed (khelsutra
#117). Follow-ups: `api.` CNAME/CF-Access cleanup. Pairs [[compute-stack-gcp-only]] [[ui-driven-
cloud-serving-gap]].
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/session-
handoff.md:19:- **khelsutra site-droplet RETIRED** ([[khelsutra-site-droplet-parity]]): the
dual-purpose `claude-do-rally-test`/`168.144.126.176` (marketing-site mirror + alpha engine)
kept drifting from CF ? **deleted ($0; data archived to `gs://khelsutra-rally-
corpus/archive/droplet-?-20260625/` + local), parity monitor removed (khelsutra #117), Node-20
actions bumped (#118).** ? This means the prior "LIVE alpha on the droplet" in the Earlier block
is **GONE** ? alpha is paused pending the Cloud-Run move; `api.khelsutra.guru` CNAME/CF-Access
need cleanup.
/c/Users/avidu/.claude/projects/c--Users-avidu-Projects-khelsutra-guru/memory/session-
handoff.md:28:### Earlier ? serving/alpha track (2026-06-22/23, UNCHANGED + owner-gated; note
the droplet alpha is now retired per above)

```

28. user (2026-07-02T02:50:16.737Z)

```

Name:    api.khelsutra.guru
Addresses: 2606:4700:3035::6815:4f4f
          2606:4700:3037::ac43:a95b
          104.21.79.79
          172.67.169.91

```

```

----
Name:    app.khelsutra.guru
Addresses: 2606:4700:3035::6815:4f4f
          2606:4700:3037::ac43:a95b
          172.67.169.91
          104.21.79.79

```

29. user (2026-07-02T02:50:46.092Z)

```

105:/** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
107: return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid });
172:/** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 + a
job_id;
173: * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
177:export async function compileReel(
238:/** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */

```

30. user (2026-07-02T02:50:48.283Z)

```

12:dependency-free **Node + SQLite** host (`site/server.js`), **live + browsable on the DO
droplet ?
13:http://168.144.126.176:8787** (systemd `khelsutra-site`). Shipped across ~90 merged PRs (the
form,
14:6-language localization, the per-rally MVP, droplet portability). Docs:
64: (`deploy-test@`, `email-test@`). Revoke the droplet SSH key (`~/.ssh/khelsutra_droplet`)
when done.
77: profile of the engine's decoupled `StorageBackend` + `backend.worker` architecture. MVP

```

```

runs on the CPU droplet via
78: **Gemini/stub**; **real local-GPU detection needs a GPU host** (DO droplet / GCE VM / Cloud
Run+GPU ? owner priority
=====
./deploy/DEPLOY_ALPHA.md:35:- A **box** to run the engine: the CPU droplet `168.144.126.176`
(Gemini-only ? no GPU; fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` +
Python 3.8+ installed; the `badminton-highlight-indexer` engine checked out.
./deploy/droplet/README.md:4:(`168.144.126.176`, Gemini, **no GPU**). Values are pre-baked for
**`app.khelsutra.guru` /
./deploy/droplet/README.md:87:curl --max-time 5 http://168.144.126.176:8000/api/health      #
-> refused / timeout
./deploy/droplet/README.md:100:> **? Live-verified on the droplet (2026-06-22).** Engine
installed + booted on `168.144.126.176`; full
./docs/DRY_RUN_CUJ.md:42:3. **Engine host:** local Windows (simplest, Gemini/motion) . the CPU
droplet `168.144.126.176` (shared testbed ? careful) . a GPU VM (only for WASB).
./docs/LOCAL_APPLIANCE_ARCHITECTURE.md:48:- **Droplet reality:** `168.144.126.176`, **CPU-only,
NO GPU**, region `blr1`; co-hosts the marketing site (`/srv/khelsutra-site`, node `:8787`)
**and** the engine (`~/rally`); ran the full train+infer e2e against DO Spaces with the **GPU
mocked** `[SETUP_NEW_MACHINE.md, DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO
`9000/9001` + GCS emulator `9023` also run there.
./docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176:*** Alpha deploy path chosen = SPLIT, not the single-
origin shell (owner 2026-06-21).** The alpha ships the **split** topology (SPA on Cloudflare
Pages + engine on an `api.*` cloudflared tunnel + CF Access over both ? the engine
`HOSTING_PLAN.md` §2), **not** the single-origin Caddy shell of **T2** (kept as the documented
alternative ? §0/D1). The **deploy kit + runbook**
(`[`deploy/DEPLOY_ALPHA.md`]`(`./deploy/DEPLOY_ALPHA.md`) +
`deploy/cloudflared.config.example.yml`) is shipped (? templates+runbook). **The alpha stand-up
is now LIVE (2026-06-22):** the engine booted on the droplet (`168.144.126.176`),
`app.khelsutra.guru` is up, and the full **DNS ? Cloudflare ? Access ? tunnel ? engine** chain
plus the **T3** raw-`:8000`-refused posture test (boot-posture fail-fast, Host-guard `400?200`,
path-jail, T4 upload?jail, edge-auth `401`-gate) were all **exercised live**
`[`deploy/droplet/README.md:100-106`]`. Only the **Gemini-key rotation** + a **browser OTP login**
remain. Enabled by: engine exposure-safety boot posture (engine #294) + SPA `VITE_API_BASE`
(`[#64])`.
./NEXT_STEPS.md:13:http://168.144.126.176:8787** (systemd `khelsutra-site`). Shipped across ~90
merged PRs (the form,

```

31. user (2026-07-02T02:51:06.061Z)

```

160     body: JSON.stringify({ video_id: videoId }),
161     cid,
162   });
163   return data.play_segments ?? [];
164 }
165
166   // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
167   const COMPILE_POLL_MIN_MS = 1000;
168   const COMPILE_POLL_MAX_MS = 5000;
169   const COMPILE_POLL_BACKOFF = 1.5;
170   const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
forever
171
172   /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
a job_id;
173   * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
174   * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
built. Resolves
175   * with the job's CompileResult on success; throws ApiError on a failed/timed-out
compile.
176   * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
absent). */
177   export async function compileReel(
178     videoId: string,

```

```

179     segmentIds: number[],
180     outputFilename: string,
181     cid?: string,
182     locale?: string,
183 ): Promise<CompileResult> {
184     const c = cid ?? newCorrelationId();
185     const submitted = await request<ProcessResponse>("/compile", {
186         method: "POST",
187         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
188         cid: c,
189     });
190     return pollCompileJob(submitted.job_id, c);
191 }
192
193 /** Poll a submitted compile job to its terminal state. Returns the reel result on
success; throws an
194 * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
failed / no URL),
195 * or a poll timeout ? so the caller's existing classifyError/errorText mapping
surfaces it like ingest. */
196 async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
197     const startedAt = Date.now();
198     let delay = COMPILE_POLL_MIN_MS;
199     for (;;) {
200         const job = await getJob(jobId, cid);
201         if (job.status === "done") {
202             const result = (job.result ?? {}) as CompileResult;
203             if (result.status === "failed" || !result.download_url) {
204                 // A real stitch failure carries an engine message ? surface it as an `engine`
detail (status +
205                 // body set) so classifyError shows it verbatim, NOT the no-status "engine
unreachable" fallback.
206                 const msg = result.message || "Highlight stitching failed.";
207                 throw new ApiError(msg, { cid, status: 502, body: msg });
208             }
209             return result;
210         }
211         if (job.status === "failed") {
212             const msg = job.error || "Compile job failed.";
213             throw new ApiError(msg, { cid, status: 502, body: msg });
214         }
215         if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
216             throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
217         }
218         await new Promise((resolve) => setTimeout(resolve, delay));
219         delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
220     }
221 }
222
223 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
224 export function highlightUrl(videoId: string, filename: string): string {
225     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
226 }
227
228 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
229 export async function getConfig(cid?: string): Promise<EngineConfig> {
230     return request<EngineConfig>("/config", { method: "GET", cid });
231 }
232
233 /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */

```

```
234     export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
```

32. assistant (2026-07-02T02:51:48.433Z)

All 10 issues triaged and verified against code. Quick CI-state check for the summary, then I'll report.

33. user (2026-07-02T02:51:50.705Z)

```
completed      success web: resilient video-summary readers + show fps in the library (#140)
test   master  push    28532935786      1m28s   2026-07-01T16:38:03Z
completed      success docs: refresh khelsutra docs for today's storage-track work (P2-P5) (?)
test   master  push    28530208455      1m29s   2026-07-01T15:51:14Z
completed      success web: persistent "FFmpeg not detected" banner (hard-blocker warning) (?)
test   master  push    28530204391      1m14s   2026-07-01T15:51:10Z
completed      success P5: SPA storage-medium picker (store a video into your medium) (#137)
test   master  push    28525000285      1m24s   2026-07-01T14:28:57Z
completed      success docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
test   master  push    28521285119      1m28s   2026-07-01T13:30:54Z
```

34. user (2026-07-02T02:52:57.961Z)

Structured output provided successfully